

Fast Fully Oblivious Sampling

Michael Shell
School of Electrical and
Computer Engineering
Georgia Institute of Technology
Atlanta, Georgia 30332-0250

Email: <http://www.michaelshell.org/contact.html>

Homer Simpson
Twentieth Century Fox
Springfield, USA
Email: homer@thesimpsons.com San Francisco, California 96678-2391
James Kirk
and Montgomery Scott
Starfleet Academy
Telephone: (800) 555-1212
Fax: (888) 555-1212

Abstract—The abstract goes here.

1. Introduction

Oblivious algorithms are a fundamental building block for privacy-preserving computation over sensitive data. Trusted execution environments (TEEs), such as Intel SGX, protect the confidentiality and integrity of data during execution, but do not inherently hide data-dependent memory accesses. An adversarial operating system may therefore infer sensitive information from observable access patterns even when the underlying data remain encrypted or protected inside the enclave [1], [2]. Oblivious algorithms address this leakage by ensuring that memory-access and execution patterns are independent of secret data. They have consequently become important building blocks in TEE-based systems, including oblivious sorting and shuffling [3], [4], oblivious data structures and search [5], and privacy-preserving machine learning [6]. As these systems increasingly process large datasets, the concrete efficiency of such oblivious primitives directly affects end-to-end system performance.

Among these primitives, *sampling* is particularly important for data-intensive workloads that operate on small subsets of large datasets. Random sampling is widely used in approximate query processing [7], [8], statistical estimation and bootstrapping [9], and mini-batch machine learning [9], [10]. In many of these applications, the goal is not to generate a single sample, but to repeatedly draw multiple independent samples from the same dataset. For example, bootstrapping repeatedly evaluates a computation over independently drawn samples, while model training may consume a large sequence of mini-batches. At the same time, sample membership itself may be sensitive: directly fetching the selected records reveals their identities through memory locations and can invalidate privacy guarantees that rely on hidden sampling [9]. An efficient TEE therefore needs not only oblivious sampling, but an efficient way to generate many independent samples while hiding all sample memberships.

Unfortunately, generating multiple oblivious samples remains expensive. A straightforward approach executes an

oblivious sampling procedure independently for every sample, repeatedly incurring dataset-scale processing even when each sample is small. Prior work has therefore turned to *batch sampling* to amortize this cost. PSQF generate multi-sample sampling-without-replacement samples using two oblivious shuffles and a replication scan [9], while MASQUE performs secure batch sampling in the MPC setting to amortize input-scale secure computation across multiple samples [8]. These results demonstrate that batching is essential, but they also leave substantial room for improving the data-movement cost in TEE-based execution. Global oblivious shuffles are expensive, while independently routing each sample still repeats work across samples drawn from the same input. Moreover, independent samples may overlap, so an efficient batch algorithm must not only route records to different outputs, but also obviously replicate records that belong to multiple samples without revealing their memberships. The central challenge is therefore to share as much routing and replication work as possible across the batch while preserving both sample independence and full obliviousness.

In this paper, we explore the following question:

Can we achieve concretely efficient, fully oblivious batch sampling that jointly generates independent uniform samples without replacement while substantially reducing redundant data movement?

2. Preliminaries

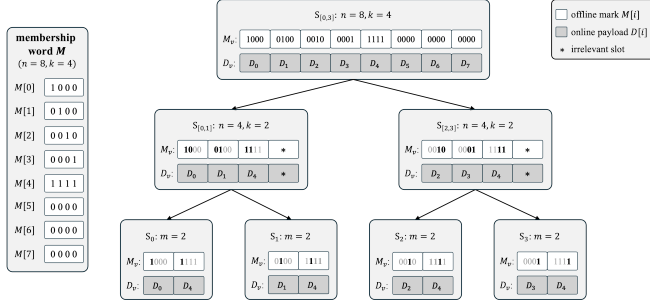


Figure 1. Hierarchical binary-tree sampling framework shared by FFOS-C and FFOS-FR. Each node represents a group of samples and its corresponding candidate array, whose public capacity is determined by the number of descendant samples. Membership words M_v are processed offline to determine how records propagate to the child subtrees, while payload records D_v follow the same tree online using the precomputed controls. A record may propagate to either child or to both children when it belongs to samples in different descendant groups. Membership bits are shown from left to right in the order $S_0, S_1, \dots, S_{k-1}$.

3. FFOS-C: Compaction-Based Fast Fully Oblivious Sampling

3.1. Design Overview

Our sampling algorithm is built around the binary sampling tree illustrated in Figure 1. The root represents all k samples, each internal node represents a group of samples and divides it into two approximately equal subgroups, and each leaf corresponds to one final sample. Each node maintains a candidate array containing the records that may still be needed by at least one sample in its subtree. As the traversal proceeds from the root toward the leaves, sample groups are progressively separated and the corresponding candidate arrays become smaller.

To determine how records propagate through the tree, we associate each input record $D[i]$ with a k -bit membership word $M[i]$. Bit j of $M[i]$ indicates whether $D[i]$ belongs to sample S_j . Hence, for a node representing a sample group G_v , a record is relevant to that node whenever at least one membership bit corresponding to G_v is set. At an internal node, a record may therefore be needed by the left child, the right child, both children, or neither child. In particular, samples are independent but need not be disjoint, so a record selected by samples in both child groups must propagate to both subtrees.

The tree also bounds the amount of data that must be retained at each node. If node v represents k_v samples, these samples contain at most mk_v distinct input records in total. Thus, if the parent candidate array has length n_{par} , the public capacity of node v is $n_v = \min\{n_{\text{par}}, mk_v\}$. Only the first n_v positions need to be retained for the subtree. If fewer than n_v records are actually relevant, the retained prefix may additionally contain irrelevant records. The retained length therefore depends only on public parameters and not on the actual overlap among the descendant samples. At a leaf, $k_v = 1$ and the candidate-array capacity becomes m , so the m relevant records form the final sample.

Algorithm 1 FFOS-C(D, n, m, k)

Input: Secret data array D of public length n ; public $1 \leq m < n$ and $k > 1$.

Output: k independent uniform size- m samples.

Offline phase

- 1: $M \leftarrow \text{MARKSAMPLES}(n, m, k)$
- 2: $C \leftarrow \text{FFOSCCONTROLBITS}(M, n, m, k)$

Online phase

- 3: $(S, p) \leftarrow \text{FFOSCAPPLY}(D, C, n, m, k, 0)$
- 4: **assert** $p = |C|$
- 5: **return** S

Figure 1 also illustrates a second key idea: the same sampling tree is used in both the offline and online phases. Sampling membership depends only on public input positions and fresh randomness, rather than on the payload values. The offline phase therefore operates only on lightweight membership information, propagating membership words through the tree and deriving the secret controls required at each node. The online phase later follows the same public tree structure and replays these controls on the actual records. In this way, sampling-dependent decisions are separated from payload processing: the offline phase determines how records should move, whereas the online phase only executes the predetermined oblivious operations.

The binary-tree framework specifies which candidate records each child node must receive, but does not prescribe how the two child candidate arrays are obviously materialized. FFOS-C realizes this node operation using oblivious compaction: for each child group, it compacts the records relevant to that group to the front and retains the corresponding public-length prefix. FFOS-FR, introduced in Section 4, uses the same sampling tree but realizes suitable node splits with oblivious fork routing. The remainder of this section focuses on the compaction-based realization.

Notation. Arrays use zero-based indexing. We write $A[i]$ for the element of A at index i , $|A|$ for its length, and $A[a : b)$ for the half-open subarray $(A[a], \dots, A[b - 1])$. We write $\llbracket Q \rrbracket$ for the obviously computed indicator of predicate Q , equal to 1 if Q holds and 0 otherwise. The operators $\&$ and $|$ denote bitwise AND and OR, respectively, and $\ll$ and $\gg$ denote left and right bit shifts. We use $\parallel$ for ordered concatenation. These primitives can be implemented efficiently and obviously using boolean arithmetic or conditional instructions such as CMOVZ.

3.2. Offline Control Generation

In FFOS-C, each child candidate array in the sampling tree is materialized using oblivious compaction. A compaction therefore requires, for every position in the current candidate array, a secret bit indicating whether the corresponding record is needed by at least one sample in the child subtree. Since these decisions depend only on sample membership rather than on the payload values, they can be prepared before the data records are available.

Algorithm 2 MARKSAMPLES(n, m, k)

Input: Public n, m, k with $1 \leq m \leq n$.**Output:** Secret length- n membership array M for k size- m samples.

```
1: Create a length- $n$  array  $M$  initialized to 0.
2: Create a length- $k$  array  $R$  initialized to  $m$ .
3: for  $i \leftarrow 0 \dots n-1$  do
4:   for  $j \leftarrow 0 \dots k-1$  do
5:     Generate  $\rho \in [0, 1)$  uniformly at random.
6:      $u \leftarrow \lfloor \rho < R[j]/(n-i) \rfloor$ 
7:      $M[i] \leftarrow M[i] \mid (u \ll j)$ 
8:      $R[j] \leftarrow R[j] - u$ 
9:   end for
10: end for
11: return  $M$ 
```

The offline phase first generates the membership array M using MARKSAMPLES (Algorithm 2), and then transforms these memberships into the compaction controls required throughout the sampling tree using FFOSCCONTROLBITS (Algorithm 3).

MARKSAMPLES independently generates a uniform size- m subset of the n input positions for each sample. For sample j , it maintains a secret remaining quota $R[j]$ and selects position i with probability $R[j]/(n-i)$. This sequential rule produces exactly m positions uniformly without replacement. Fresh independent randomness across samples makes the corresponding columns of M independent, while still allowing different samples to overlap. Thus, M jointly captures the sampling decisions for all k samples with respect to the original input array.

The original membership array, however, is not sufficient to derive all controls directly. At the root, its membership words immediately determine which input positions are needed by each child sample group. After this first split, however, each child operates on a candidate array whose records have been reordered by compaction and whose public length may have been reduced. Controls for deeper nodes must therefore be derived with respect to the local record order produced by their ancestors, rather than the original input order.

To achieve this, FFOSCCONTROLBITS propagates the membership information through the same tree transformations that the data records will later undergo online. For every node v , the resulting membership array M_v is kept positionally aligned with the corresponding future candidate array D_v : $M_v[i]$ describes the record that will occupy $D_v[i]$. The membership bits associated with each child determine its compaction flags, and applying these flags to the membership words produces the membership state for that child. Because the same flags induce the same compaction permutation independently of the array contents, replaying those controls on the data records preserves this correspondence at every level. Retaining the public prefix $n_v = \min\{n, mk_v\}$ is sufficient because the k_v descendant samples contain at most mk_v distinct selected positions.

Repeating this process recursively converts the global

Algorithm 3 FFOSCCONTROLBITS(M, n, m, k)

Input: Secret membership array M ; public n, m, k .**Output:** Secret compaction-control array C .

```
1: if  $k = 1$  then
2:   assert  $n = m$ 
3:   return  $\emptyset$ 
4: end if
5:  $k_l \leftarrow \lfloor k/2 \rfloor$ ,  $k_r \leftarrow k - k_l$ 
6:  $\mu_l \leftarrow (1 \ll k_l) - 1$ ,  $\mu_r \leftarrow (1 \ll k_r) - 1$ 
7: Create bit arrays  $C_l, C_r$ , each of length  $n$ .
8: for  $i \leftarrow 0 \dots n-1$  do
9:    $C_l[i] \leftarrow \lfloor (M[i] \& \mu_l) \neq 0 \rfloor$ 
10:   $C_r[i] \leftarrow \lfloor ((M[i] \gg k_l) \& \mu_r) \neq 0 \rfloor$ 
11: end for
12:  $n_l \leftarrow \min(n, mk_l)$ ,  $n_r \leftarrow \min(n, mk_r)$ 
13: Copy  $M$  into independent arrays  $M_l$  and  $M_r$ .
14: OCOMPACT( $M_l, C_l$ )
15: OCOMPACT( $M_r, C_r$ )
16: for  $i \leftarrow 0 \dots n_l-1$  do
17:    $M_l[i] \leftarrow M_l[i] \& \mu_l$ 
18: end for
19: for  $i \leftarrow 0 \dots n_r-1$  do
20:    $M_r[i] \leftarrow (M_r[i] \gg k_l) \& \mu_r$ 
21: end for
22:  $M_l \leftarrow M_l[0 : n_l]$ ,  $M_r \leftarrow M_r[0 : n_r]$ 
23:  $C_l^{\text{sub}} \leftarrow \text{FFOSCCONTROLBITS}(M_l, n_l, m, k_l)$ 
24:  $C_r^{\text{sub}} \leftarrow \text{FFOSCCONTROLBITS}(M_r, n_r, m, k_r)$ 
25: return  $C_l \parallel C_r \parallel C_l^{\text{sub}} \parallel C_r^{\text{sub}}$ 
```

membership decisions in M into the local compaction controls required by all internal nodes. These controls are stored in a fixed traversal order determined only by the public sampling tree; their values remain secret. Once the control array has been generated, the membership information is no longer needed online. FFOSCAPPLY simply consumes the precomputed controls in the same tree order and applies them to the actual records, as described in Section 3.3.

3.3. Online Sampling Execution

Once the payload records become available, FFOSCAPPLY (Algorithm 4) replays the controls generated offline on the actual data. It traverses the same public sampling tree as FFOSCCONTROLBITS and, at each internal node, applies the corresponding selection flags to independent copies of the current candidate array using oblivious compaction. Each child retains its predetermined public-length prefix and continues recursively. Because the offline phase generated descendant controls using the same sequence of compactations, these controls refer to exactly the record positions encountered online. No membership information is accessed during this phase; all sampling-dependent decisions have already been encoded in C . When a leaf is reached, its m records directly constitute the corresponding sample.

Example. Figure 1 illustrates the complete process for $n = 8$, $m = 2$, and $k = 4$. The membership words shown at

Algorithm 4 FFOSCAPPLY(D, C, n, m, k, p)

Input: Secret membership array M ; public n, m, k .**Output:** Secret compaction-control array C .

```
1: if  $k = 1$  then
2:   assert  $n = m$ 
3:   Copy  $D[0 : m)$  into  $S$ .
4:   return  $([S], p)$ 
5: end if
6:  $k_l \leftarrow \lfloor k/2 \rfloor$ ,  $k_r \leftarrow k - k_l$ 
7:  $C_l \leftarrow C[p : p + n)$ ,  $p \leftarrow p + n$   $\triangleright$  Control view
8:  $C_r \leftarrow C[p : p + n)$ ,  $p \leftarrow p + n$   $\triangleright$  Control view
9:  $n_l \leftarrow \min(n, mk_l)$ ,  $n_r \leftarrow \min(n, mk_r)$ 
10: Copy  $D$  into independent arrays  $D_l$  and  $D_r$ .
11: OCOMPACT( $D_l, C_l$ )
12: OCOMPACT( $D_r, C_r$ )
13:  $D_l \leftarrow D_l[0 : n_l)$ ,  $D_r \leftarrow D_r[0 : n_r)$ 
14:  $(S_l, p) \leftarrow \text{FFOSCAPPLY}(D_l, C, n_l, m, k_l, p)$ 
15:  $(S_r, p) \leftarrow \text{FFOSCAPPLY}(D_r, C, n_r, m, k_r, p)$ 
16: return  $(S_l \parallel S_r, p)$ 
```

the root encode the four sample sets. For example, $M[4] = 1111$ means that D_4 belongs to all four samples and must therefore propagate to both root subtrees, whereas $M[5] = 0000$ means that D_5 is irrelevant to all samples.

At the root, the four samples are divided into the left group (S_0, S_1) and the right group (S_2, S_3) . To construct the left child, FFOS-C sets the control bit of position i to 1 iff D_i is selected by at least one sample in (S_0, S_1) , i.e., iff either of the first two bits of $M[i]$ is 1. For example, $M[0] = 1000$ gives control bit 1, $M[2] = 0010$ gives 0, and $M[4] = 1111$ gives 1. Applying this rule to all eight positions gives $C_l = (1, 1, 0, 0, 1, 0, 0, 0)$, which selects D_0, D_1, D_4 . The left child has public capacity $n_l = \min\{8, 2 \cdot 2\} = 4$. After offline compaction, its retained membership words are $(1000, 0100, 1111, *)$, corresponding online to the candidate array $(D_0, D_1, D_4, *)$. The right child is formed analogously using $C_r = (0, 0, 1, 1, 1, 0, 0, 0)$, retaining $(0010, 0001, 1111, *)$ and the corresponding candidate array $(D_2, D_3, D_4, *)$.

At the left child, the two samples are separated. The first membership bit gives the control for S_0 as $(1, 0, 1, 0)$, while the second gives the control for S_1 as $(0, 1, 1, 0)$. Replaying these controls online produces $S_0 = \{D_0, D_4\}$ and $S_1 = \{D_1, D_4\}$. The right subtree is processed analogously, producing $S_2 = \{D_2, D_4\}$ and $S_3 = \{D_3, D_4\}$.

3.4. Efficiency Optimizations

The public sampling-tree structure also allows the control storage to be determined before control generation begins. Since the tree topology, candidate-array capacities, and the number of selection bits required at every node depend only on the public parameters n, m , and k , the total control-array length can be computed in advance. FFOS-C can therefore allocate a single contiguous array C and assign each subtree a public, disjoint segment of this array.

Algorithm 5 BUILDFRONTIER(n, m, k)

Input: Public n, m, k .**Output:** Ordered frontier $F = [(s_v, k_v)]$.

```
1:  $q \leftarrow \lfloor n/m \rfloor$   $\triangleright$  Maximum number of samples per frontier node
2:  $r \leftarrow \lceil k/q \rceil$   $\triangleright$  Minimum number of frontier nodes
3:  $b \leftarrow \lfloor k/r \rfloor$ 
4:  $t \leftarrow k \bmod r$   $\triangleright$  First  $t$  nodes contain one additional sample
5: Create an empty frontier array  $F$ 
6:  $s \leftarrow 0$ 
7: for  $v \leftarrow 0 \dots r - 1$  do
8:    $k_v \leftarrow b + 1$  if  $v < t$ , otherwise  $k_v \leftarrow b$ 
9:   Append  $(s, k_v)$  to  $F$ ;  $s \leftarrow s + k_v$ 
10: end for
11: return  $F$ 
```

Control generation writes directly into these segments, and the online phase later reads them in the same fixed traversal order. This avoids repeated allocation and concatenation of intermediate control arrays and improves memory locality during both offline and online execution.

When $mk > n$, the upper levels of the binary sampling tree may perform substantial redundant work. For a node representing k_v samples, its candidate capacity is $n_v = \min\{n, mk_v\}$; hence, whenever $mk_v \geq n$, splitting the sample group does not yet reduce the candidate-array size. Several consecutive upper levels may therefore repeatedly process full-length arrays before any reduction occurs. To bypass these non-shrinking levels, we introduce a public *frontier* from which the binary recursion begins. Let $q = \lfloor n/m \rfloor$ be the maximum number of samples that can fit in one frontier node and $r = \lceil k/q \rceil$ the minimum number of frontier nodes. BUILDFRONTIER (Algorithm 5) partitions the k samples into r consecutive groups of nearly equal sizes, each satisfying $mk_v \leq n$. Each frontier group is then processed from the original input array to form a candidate array of public capacity mk_v , after which the ordinary binary recursion proceeds within that group.

The same tree structure also exposes natural parallelism in both phases. Once an internal node has prepared the inputs and control-array segments for its two children, the left and right subtrees are independent and can be processed concurrently. This recursive independence provides parallelism at every level of the sampling tree. The frontier further increases the available coarse-grained parallelism: its $r = |F|$ groups can be processed independently from the outset, while each group subsequently exposes additional parallelism through its own binary recursion. The actual degree of concurrency can therefore be adapted to the available cores, memory capacity, and bandwidth without changing the public execution structure or the resulting samples.

3.5. Analysis

We now analyze the correctness, obliviousness, and efficiency of FFOS-C. We first show that the offline-generated

controls faithfully realize the memberships produced by MARKSAMPLES, and therefore yield independent uniform samples without replacement. We then establish that both the offline control-generation phase and the online execution phase are fully oblivious to their secret inputs. Finally, we analyze the control complexity and the offline and online runtime of FFOS-C. Detailed proofs of the following theorems are deferred to the appendix A.

3.5.1. Correctness. We establish the correctness of FFOS-C in three steps. We first show that MARKSAMPLES generates the desired sampling memberships, then show that the recursive control generation and online execution faithfully realize these memberships, and finally derive the correctness of the complete FFOS-C algorithm.

Theorem 1. *Let $M = \text{MARKSAMPLES}(n, m, k)$, and let $M^{(j)} = (M[0][j], \dots, M[n-1][j])$ denote the membership vector of sample j . Let $\text{wt}(x)$ denote the number of 1 bits in a bit vector x . Then $M^{(0)}, \dots, M^{(k-1)}$ are mutually independent, and each $M^{(j)}$ is uniformly distributed over $\{x \in \{0, 1\}^n : \text{wt}(x) = m\}$.*

Proof Intuition. For each sample, MARKSAMPLES maintains the number of records that remain to be selected and processes the input positions sequentially. At each position, the selection probability equals the ratio between the remaining quota and the number of remaining positions. Consequently, every size- m subset is selected with the same probability. Independent randomness across samples makes the resulting membership vectors mutually independent.

Theorem 2. *Let M be a length- n membership array, and let $M^{(j)} = (M[0][j], \dots, M[n-1][j])$ denote the membership vector of sample j . Let $\text{wt}(x)$ denote the number of 1 bits in a bit vector x . Suppose that $\text{wt}(M^{(j)}) = m$ for every $j \in \{0, \dots, k-1\}$. If $C = \text{FFOSCCONTROLBITS}(M, n, m, k)$ and $(S, p) = \text{FFOSCAPPLY}(D, C, n, m, k, 0)$, then $p = |C|$ and, for every j , S_j is a permutation of the records $D[i]$ for which $M[i][j] = 1$.*

Proof Intuition. The proof maintains the invariant that, at every sampling-tree node, the membership words processed offline remain positionally aligned with the candidate records processed online. For each child, FFOSCCONTROLBITS applies the same compaction controls to the membership words that FFOSCAPPLY later applies to the corresponding records. This alignment is therefore preserved recursively until each leaf contains exactly the records selected by its membership vector.

Theorem 3. *For public parameters $1 \leq m < n$ and $k > 1$, $\text{FFOS-C}(D, n, m, k)$ outputs k mutually independent samples, where each sample is a uniformly random size- m subset of the n input records and is sampled without replacement.*

Proof Intuition. By Theorem 1, MARKSAMPLES generates k independent and uniform size- m membership selections.

By Theorem 2, FFOSCCONTROLBITS and FFOSCAPPLY faithfully realize these selections on the input records. Together, these two results imply that FFOS-C produces exactly the k independent uniform size- m samples specified by MARKSAMPLES.

3.5.2. Obliviousness. We next establish the full obliviousness of FFOS-C. We first show that the offline control-generation procedure hides the membership information from its execution trace, and then show that the online procedure hides both the payload records and the precomputed controls. Combining these results with the oblivious execution of MARKSAMPLES establishes the full obliviousness of FFOS-C.

Theorem 4. *For fixed public parameters n , m , and k , $\text{FFOSCCONTROLBITS}(M, n, m, k)$ is fully oblivious with respect to the secret membership array M .*

Proof Intuition. The recursion structure, candidate-array lengths, and loop bounds of FFOSCCONTROLBITS are determined entirely by the public parameters. Membership bits affect only secret arithmetic, bitwise operations, and the control inputs to fully oblivious compaction. Consequently, they do not affect the observable execution trace.

Theorem 5. *For fixed public parameters n , m , and k , $\text{FFOSCAPPLY}(D, C, n, m, k, 0)$ is fully oblivious with respect to the secret data array D and the control array C .*

Proof Intuition. FFOSCAPPLY follows the same public sampling-tree structure for all inputs. The sizes of all candidate arrays and control segments are fixed by n , m , and k , while the values in C are used only as secret controls to fully oblivious compaction. Hence, neither the payload values nor the sampling-dependent controls influence the observable execution trace.

Theorem 6. *For fixed public parameters n , m , and k , $\text{FFOS-C}(D, n, m, k)$ is fully oblivious with respect to the secret data array D .*

Proof Intuition. MARKSAMPLES executes a fixed number of iterations determined only by n and k , and processes its secret quotas and random sampling decisions using oblivious operations. By Theorems 4 and 5, both control generation and online execution are fully oblivious. Therefore, the complete execution of FFOS-C is fully oblivious.

3.5.3. Efficiency. We characterize the efficiency of basic FFOS-C using two quantities: the storage required for the offline-generated controls and the number of oblivious swaps performed during online execution. We instantiate OCOMPACT with ORCompact [3], for which the number of OSWAP operations provides a concrete measure of payload movement and dominates the cost for sufficiently large records.

Theorem 7. Let $B(n, m, k)$ denote the number of control bits generated by $\text{FFOSCONTROLBITS}(M, n, m, k)$. Then

$$B(n, m, 1) = 0,$$

and, for $k > 1$,

$$B(n, m, k) = 2n + B(n_l, m, k_l) + B(n_r, m, k_r),$$

where $k_l = \lfloor k/2 \rfloor$, $k_r = k - k_l$, and $n_l = \min\{n, mk_l\}$, $n_r = \min\{n, mk_r\}$. In particular, if $n = mk$ and k is a power of two, then

$$B(n, m, k) = 2n \log_2 k.$$

Proof Intuition. Every internal node with candidate-array length n_v generates two length- n_v control vectors, one for each child. When $n = mk$ and k is a power of two, the total candidate-array length at every internal level is exactly n . Hence each level contributes $2n$ control bits, over $\log_2 k$ internal levels.

Theorem 8. Let $S_C(n, m, k)$ denote the number of OSWAP operations performed by the online phase of basic FFOS-C when OCOMPACT is instantiated with ORCompact. Let $S_1(x)$ denote the number of OSWAP operations performed by ORCompact on an input of length x . Then

$$S_C(n, m, 1) = 0,$$

and, for $k > 1$,

$$S_C(n, m, k) = 2S_1(n) + S_C(n_l, m, k_l) + S_C(n_r, m, k_r),$$

where $k_l = \lfloor k/2 \rfloor$, $k_r = k - k_l$, $n_l = \min\{n, mk_l\}$, $n_r = \min\{n, mk_r\}$. In particular, if $n = mk$, $m = 2^a$, and $k = 2^h$, then

$$S_C(n, m, k) = n \left(ha + \frac{h(h+1)}{2} \right).$$

Proof Intuition. Each internal node invokes ORCompact twice on its current candidate array, once for each child. Thus, a node of length x contributes $2S_1(x)$ OSWAP operations. When $n = mk$, $m = 2^a$, and $k = 2^h$, the 2^d nodes at depth d all have length $n/2^d$, which is a power of two. Using $S_1(x) = (x/2) \log_2 x$ for ORCompact and summing over the h internal levels gives the stated result.

4. FFOS-FR: Fork-Routing-Based Fast Fully Oblivious Sampling

4.1. Structural Observation

Our starting point is a structural property of the sampling tree constructed by FFOS-C. Recall that a node v representing k_v samples maintains a candidate array of public length $n_v = \min\{n_{\text{par}}, mk_v\}$, where n_{par} is the length of its parent candidate array. We observe that once a node satisfies $n_v = mk_v$, this equality is preserved throughout its descendant subtree. Specifically, let the two children of v represent k_l and k_r samples, where $k_l + k_r = k_v$. Their candidate

capacities are $n_l = \min\{n_v, mk_l\}$, $n_r = \min\{n_v, mk_r\}$. Since $k_l, k_r < k_v$ and $n_v = mk_v$, we have $n_l = mk_l$, $n_r = mk_r$, and therefore $n_v = n_l + n_r$. Hence, once $n_v = mk_v$ holds at some node, every descendant node satisfies the same relation, while the capacity of each parent is exactly equal to the combined capacities of its two children. We refer to such nodes as *exact-fit nodes*.

This capacity relation reveals a redundancy in the basic FFOS-C construction. At a general internal node, FFOS-C creates two copies of the current candidate array and performs two independent oblivious compactations, one for each child. For an exact-fit node, however, the n_v slots already match the total number of slots required by the two children, $n_l + n_r$. From a capacity perspective, the two child candidate arrays can therefore be viewed as a partition of the parent's storage rather than as two independently materialized copies of the parent array. The remaining difficulty is that a record may be needed by both children and must consequently appear once in each child array.

Surprisingly, we observe that exact-fit nodes always contain sufficient irrelevant records to accommodate such duplication. Partition the n_v candidate records into four classes according to their relevance to the two children. Let N_L and N_R denote the numbers of records required only by the left and right child, respectively; let N_B denote the number required by both children; and let $N_\emptyset$ denote the number required by neither child. Then

$$n_v = N_L + N_R + N_B + N_\emptyset.$$

The numbers of distinct records actually required by the left and right children are $w_l = N_L + N_B$, $w_r = N_R + N_B$. Because these records must fit within the corresponding public capacities, $w_l \leq n_l$, $w_r \leq n_r$. Combining these inequalities with $n_v = n_l + n_r$ gives

$$\begin{aligned} N_\emptyset - N_B &= n_v - (N_L + N_R + 2N_B) \\ &= (n_l - w_l) + (n_r - w_r) \\ &\geq 0. \end{aligned}$$

Thus, $N_\emptyset \geq N_B$.

In other words, the number of records irrelevant to both children is always at least the number of records that must be duplicated across the two children. The unused positions occupied by these irrelevant records therefore provide enough capacity to accommodate every required duplicate without increasing the total array size.

This property becomes particularly useful after applying the frontier optimization from Section 3.4. For each frontier group containing k_v samples, BUILDFRONTIER guarantees $mk_v \leq n$, and the corresponding frontier candidate array is materialized with public length $n_v = mk_v$.

Each frontier node is therefore an exact-fit node. By the closure property above, every node in the subsequent binary recursion is also exact-fit. Consequently, once the frontier candidate arrays have been formed, the remainder of the sampling tree repeatedly satisfies the following two properties:

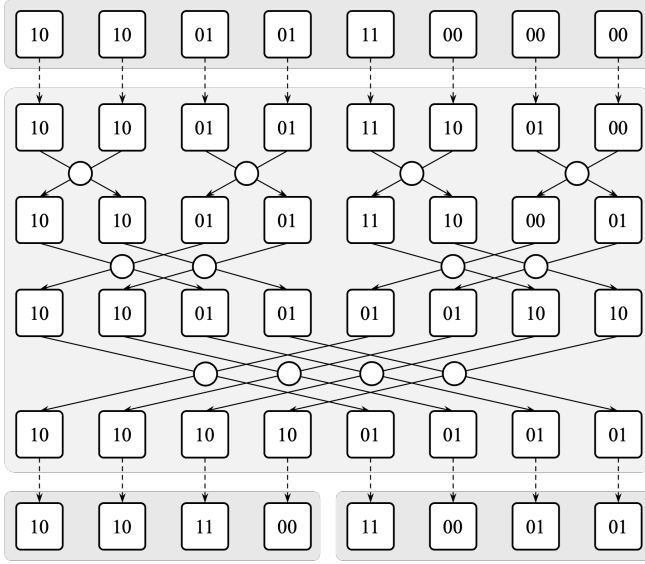


Figure 2. OFR network for $n = 8$ and $n_l = n_r = 4$. Circles denote OFORK gates with fixed butterfly-style connections.

- 1) the parent capacity equals the combined capacities of its two children, i.e., $n_v = n_l + n_r$;
- 2) the number of records irrelevant to both children is no smaller than the number of records that must be duplicated, i.e., $N_\emptyset \geq N_B$.

These observations suggest that the two independent compactions used by FFOS-C are unnecessarily general for exact-fit subtrees. Rather than duplicating the entire parent candidate array and processing the two copies independently, it may be possible to redistribute the parent array directly into its left and right output regions, duplicating only the records that are genuinely required by both children. This motivates a more specialized oblivious mechanism for processing exact-fit nodes, which we develop in the following sections.

4.2. OFR: Oblivious Fork Routing

4.2.1. Oblivious Fork (OFork). To support the joint redistribution required at exact-fit nodes, we extend oblivious swap (OSwap), a basic data-movement primitive used in practical oblivious compaction and routing algorithms [3], [11], [12], [13]. For two records x and y and a secret control bit b , let $\text{OSELECT}(x, y, b)$ return x when $b = 0$ and y when $b = 1$. We can then express OSwap as

$$\text{OSwap}(x, y, b) = (\text{OSELECT}(x, y, b), \text{OSELECT}(x, y, 1-b)).$$

The complementary controls ensure that the two outputs select different inputs, preserving one occurrence of each input record. To additionally support duplication, we allow the two outputs to select their inputs independently. For a secret control word $c = (b_0, b_1) \in \{0, 1\}^2$, we define *oblivious fork* (OFORK) as

$$\text{OFORK}(x, y, c) = (\text{OSELECT}(x, y, b_0), \text{OSELECT}(x, y, b_1)).$$

(a) OSwap	(b) OFork
<code>movq (x), %r10</code>	<code>movq (x), %r10</code>
<code>movq %r10, %r11</code>	<code>movq %r10, %r11</code>
<code>movq (y), %rcx</code>	<code>movq (y), %rcx</code>
<code>testb b, b</code>	<code>testb b0, b0</code>
<code>cmovnz %rcx, %r10</code>	<code>cmovnz %rcx, %r10</code>
<code>cmovnz %r11, %rcx</code>	<code>testb b1, b1</code>
<code>movq %r10, (x)</code>	<code>cmovnz %rcx, %r11</code>
<code>movq %rcx, (y)</code>	<code>movq %r10, (x)</code>
	<code>movq %r11, (y)</code>

Figure 3. Representative x86-64 implementations of OSwap and OFORK for an 8-byte record. Both primitives perform two CMOVZ-based oblivious selections over the same pair of inputs and the same number of payload loads and stores; OFORK differs primarily in using an independent control bit for each output.

Equivalently,

$$\text{OFORK}(x, y, c) = \begin{cases} (x, x), & c = 00, \\ (x, y), & c = 01, \\ (y, x), & c = 10, \\ (y, y), & c = 11. \end{cases}$$

Thus, 01 and 10 implement straight and cross routing, while 00 and 11 duplicate the first and second input, respectively.

This extension preserves the payload-access pattern of OSwap. Figure 3 shows representative x86-64 implementations for an 8-byte record. Both primitives use two payload loads, two conditional-move selections, and two payload stores. OFORK tests an independent control bit for each output, whereas OSwap uses a single shared test. Their similar instruction sequences suggest comparable per-operation costs, which we evaluate experimentally in Section ??.

4.2.2. OFR Network. Oblivious Fork Routing (OFR) extends this local operation to the joint redistribution of an entire candidate array. For each input record $D[i]$, we associate a secret two-bit routing tag $T[i] \in \{00, 10, 01, 11\}$, whose first and second bits indicate whether the record is required by the left and right child, respectively. Thus, 10 and 01 request only the left and right output, respectively, 11 requests both, and 00 requests neither. These tags specify the required outputs for each record; they are distinct from the control words that determine individual OFORK selections.

Let the public output capacities be n_l and n_r , with $n = n_l + n_r$ and $\sum_{i=0}^{n-1} T[i][0] \leq n_l$, $\sum_{i=0}^{n-1} T[i][1] \leq n_r$. OFR rearranges and selectively duplicates the length- n array D so that its first n_l positions form the left output and its remaining n_r positions form the right output. Each record appears once in every output region required by its tag; any remaining output positions are filled with records required by neither child. In particular, a record tagged 11 appears once in each region, using the space provided by irrelevant input records, as established for exact-fit nodes above. The order of records within either output region is unspecified.

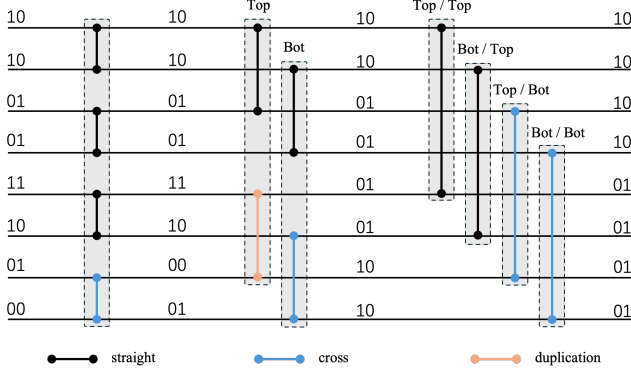


Figure 4. An illustrative responsibility assignment for the OFR network with $n = 8$ and $n_l = n_r = 4$.

TABLE 1. PAIR-ROLE CLASSIFICATION USED BY OFRASSIGN.

Tag pair	o	a_l	a_r	f_l	f_r	Tag pair	o	a_l	a_r	f_l	f_r
{00, 00}	0	0	0	0	0	{10, 01}	1	0	0	0	0
{00, 10}	0	1	0	0	0	{10, 11}	0	0	1	1	0
{00, 01}	0	0	1	0	0	{01, 01}	0	0	0	0	1
{00, 11}	0	1	1	0	0	{01, 11}	0	1	0	0	1
{10, 10}	0	0	0	1	0	{11, 11}	0	0	0	1	1

We realize this operation using a fixed network of OFORK gates with butterfly-style connections [14]. Figure 2 illustrates an eight-position instance with $n_l = n_r = 4$. Replacing the local swap operation with OFORK retains these fixed connections while enabling selective duplication within the network. The topology, including any bypasses or omitted gates, depends only on the public array length and output capacities; the routing tags affect the secret gate controls.

The network can be evaluated in place on a single length- n data array. Each gate loads its two input records and writes its two outputs back to the same pair of positions. The joint redistribution therefore uses the existing array slots without materializing two full-length copies of the candidate array. To obtain the required left and right outputs, however, the controls must coordinate selections across all stages.

4.3. OFR Control Generation and Execution

Algorithm 6 separates OFR into an offline control-generation phase and an online payload-routing phase. The offline phase derives a secret two-bit control word for every OFORK from the routing tags, while the online phase replays these controls on the aligned data array to materialize the left and right outputs. Routing tags and OFORK controls serve different purposes: the former specify which output regions a record must reach, whereas the latter determine how each individual OFORK selects its two outputs.

To bridge these two levels, we interpret each set bit of a routing tag as one *routing responsibility*. Specifically, a tag bit for the left (right) direction represents the obligation

Algorithm 6 OFR(D, T, n, n_l, n_r)

Input: Data array D , routing-tag array T ; public n, n_l, n_r with $n = n_l + n_r$.

Output: Left and right output views (D_l, D_r).

Offline phase

- 1: $\mathcal{V}_0 \leftarrow (0, 1, n, n_l, n_r)$
- 2: OFRNORMALIZE(T, n, n_l, n_r)
- 3: $C \leftarrow \text{OFRCONTROLBITS}(T, \mathcal{V}_0)$

Online phase

- 4: $p \leftarrow \text{OFRAPPLY}(D, C, \mathcal{V}_0, 0)$
 - 5: **assert** $p = |C|$
 - 6: $D_l \leftarrow D[0 : n_l], \quad D_r \leftarrow D[n_l : n]$
 - 7: **return** (D_l, D_r)
-

Algorithm 7 OFRNORMALIZE(T, n, n_l, n_r)

Input: Secret routing-tag array T ; public n, n_l, n_r .

Output: Normalize T in place to exactly n_l left and n_r right responsibilities.

- 1: $w_l \leftarrow 0, w_r \leftarrow 0$ $\triangleright$ Current left/right responsibilities
 - 2: **for** $i \leftarrow 0 \dots n - 1$ **do**
 - 3: $w_l \leftarrow w_l + T[i][0]$
 - 4: $w_r \leftarrow w_r + T[i][1]$
 - 5: **end for**
 - 6: $\delta_l \leftarrow n_l - w_l$
 - 7: $\delta_r \leftarrow n_r - w_r$
 - 8: $\rho_0 \leftarrow 0$ $\triangleright$ Number of preceding 00 tags
 - 9: **for** $i \leftarrow 0 \dots n - 1$ **do**
 - 10: $z \leftarrow \llbracket T[i] = 00 \rrbracket$
 - 11: $u_l \leftarrow z \wedge \llbracket \rho_0 < \delta_l \rrbracket$
 - 12: $u_r \leftarrow z \wedge \llbracket \delta_l \leq \rho_0 \rrbracket \wedge \llbracket \rho_0 < \delta_l + \delta_r \rrbracket$
 - 13: $T[i] \leftarrow \text{OSELECT}(T[i], 10, u_l)$
 - 14: $T[i] \leftarrow \text{OSELECT}(T[i], 01, u_r)$
 - 15: $\rho_0 \leftarrow \rho_0 + z$
 - 16: **end for**
-

to deliver the associated record to one position in the left (right) output region. Thus, tags 10 and 01 carry one left and one right responsibility, respectively, tag 11 carries both, and tag 00 carries neither. The two responsibilities of a 11-tagged record may remain together through intermediate layers and are separated only when the record is duplicated. Control generation therefore amounts to progressively assigning these responsibilities through the OFR network until the desired output tags of every OFORK are determined.

4.3.1. Tag Normalization. The original routing tags encode only the records required by the sampling task and therefore need not account for every position in the two public output regions. In particular, records tagged 00 are irrelevant to both outputs and carry no routing responsibility. Nevertheless, each such record still occupies a position in the fixed OFR network and must eventually be routed to either the left or the right output as an irrelevant filler.

We therefore normalize the routing tags before generating the network controls. OFRNORMALIZE (Algorithm 7) assigns the remaining left and right responsibilities to suit-

Algorithm 8 OFRASSIGN($T, \mathcal{V}$)

Input: Mutable secret routing-tag buffer T ; public recursive view $\mathcal{V} = (a, s, n, n_l, n_r)$.

Output: Secret current-layer control array C_{cur} .

▷ *Determine top-subnetwork responsibility targets*

```

1:  $\ell_g \leftarrow \lfloor n/2 \rfloor$ ,  $n_l^{\text{top}} \leftarrow \lceil n/2 \rceil$ 
2:  $n_l^{\text{top}} \leftarrow \lceil n_l/2 \rceil$ ,  $n_r^{\text{top}} \leftarrow n^{\text{top}} - n_l^{\text{top}}$ 
3:  $b_l \leftarrow 0$ ,  $b_r \leftarrow 0$ 
4: if  $n \bmod 2 = 1$  then
5:    $b_l \leftarrow T[a + (n-1)s][0]$ 
6:    $b_r \leftarrow T[a + (n-1)s][1]$ 
7: end if
8:  $\tau_l \leftarrow n_l^{\text{top}} - b_l$ ,  $\tau_r \leftarrow n_r^{\text{top}} - b_r$ 
  ▷ Count pair roles in the current layer
9: Initialize counters  $c_o$ ,  $c_{a_l}$ ,  $c_{a_r}$ ,  $c_{f_l}$ , and  $c_{f_r}$  to 0
10: for  $i \leftarrow 0 \dots \ell_g - 1$  do
11:    $j_0 \leftarrow a + 2 * i * s$ ,  $j_1 \leftarrow j_0 + s$ 
12:    $(o, a_l, a_r, f_l, f_r) \leftarrow \text{PAIRROLE}(T[j_0], T[j_1])$ 
13:    $c_o \leftarrow c_o + o$ 
14:    $c_{a_l} \leftarrow c_{a_l} + a_l$ ,  $c_{a_r} \leftarrow c_{a_r} + a_r$ 
15:    $c_{f_l} \leftarrow c_{f_l} + f_l$ ,  $c_{f_r} \leftarrow c_{f_r} + f_r$ 
16: end for
  ▷ Compute balancing quotas
17:  $q_o^{\min} \leftarrow \max\{0, \tau_l - c_{f_l} - c_{a_l}, c_{f_r} + c_o - \tau_r\}$ 
18:  $q_o^{\max} \leftarrow \min\{c_o, \tau_l - c_{f_l}, c_{f_r} + c_o + c_{a_r} - \tau_r\}$ 
19:  $q_o \leftarrow \min\{q_o^{\max}, \max\{q_o^{\min}, \lfloor c_o/2 \rfloor\}\}$ 
20:  $q_l \leftarrow \tau_l - c_{f_l} - q_o$ 
21:  $q_r \leftarrow \tau_r - c_{f_r} - c_o + q_o$ 
  ▷ Assign pairs and generate current-layer controls
22: Create a length- $\ell_g$  two-bit control array  $C_{\text{cur}}$ 
23:  $\rho_o \leftarrow 0$ ,  $\rho_l \leftarrow 0$ ,  $\rho_r \leftarrow 0$ 
24: for  $i \leftarrow 0 \dots \ell_g - 1$  do
25:    $j_0 \leftarrow a + 2is$ ,  $j_1 \leftarrow j_0 + s$ 
26:    $(o, a_l, a_r, f_l, f_r) \leftarrow \text{PAIRROLE}(T[j_0], T[j_1])$ 
27:    $\chi_o \leftarrow o \wedge \llbracket \rho_o < q_o \rrbracket$ 
28:    $\chi_l \leftarrow a_l \wedge \llbracket \rho_l < q_l \rrbracket$ 
29:    $\chi_r \leftarrow a_r \wedge \llbracket \rho_r < q_r \rrbracket$ 
30:    $t_l^{\text{top}} \leftarrow f_l \vee \chi_o \vee \chi_l$ 
31:    $t_r^{\text{top}} \leftarrow f_r \vee (o \wedge \neg \chi_o) \vee \chi_r$ 
32:    $t_l^{\text{bot}} \leftarrow T[j_0][0] + T[j_1][0] - t_l^{\text{top}}$ 
33:    $t_r^{\text{bot}} \leftarrow T[j_0][1] + T[j_1][1] - t_r^{\text{top}}$ 
34:    $t^{\text{top}} \leftarrow (t_l^{\text{top}}, t_r^{\text{top}})$ 
35:    $t^{\text{bot}} \leftarrow (t_l^{\text{bot}}, t_r^{\text{bot}})$ 
36:    $C_{\text{cur}}[i] \leftarrow \text{OForkCtrl}(T[j_0], T[j_1], t^{\text{top}}, t^{\text{bot}})$ 
37:    $(T[j_0], T[j_1]) \leftarrow (t^{\text{top}}, t^{\text{bot}})$ 
38:    $\rho_o \leftarrow \rho_o + o$ ,  $\rho_l \leftarrow \rho_l + a_l$ ,  $\rho_r \leftarrow \rho_r + a_r$ 
39: end for
40: return  $C_{\text{cur}}$ 

```

able 00-tagged records while leaving every original nonzero tag unchanged. The exact-fit property established in Section ?? guarantees that enough irrelevant records are available for this purpose. After normalization, the tag array satisfies $\sum_i T[i][0] = n_l$, $\sum_i T[i][1] = n_r$.

Hence, the number of left and right responsibilities exactly matches the corresponding output capacities. This

Algorithm 9 OFRCONTROLBITS($T, \mathcal{V}$)

Input: Secret routing-tag buffer T and public recursive view $\mathcal{V} = (a, s, n, n_l, n_r)$.

Output: Secret OFR control array C .

```

1:  $(a, s, n, n_l, n_r) \leftarrow \mathcal{V}$ 
2: if  $n_l = 0$  or  $n_r = 0$  then
3:   return  $[]$ 
4: end if
5: if  $n = 2$  then
6:    $c \leftarrow \text{OForkCtrl}(T[a], T[a+s], 10, 01)$ 
7:    $(T[a], T[a+s]) \leftarrow (10, 01)$ 
8:   return  $[c]$ 
9: end if
10:  $n^{\text{top}} \leftarrow \lceil n/2 \rceil$ ,  $n^{\text{bot}} \leftarrow n - n^{\text{top}}$ 
11:  $n_l^{\text{top}} \leftarrow \lceil n_l/2 \rceil$ ,  $n_r^{\text{top}} \leftarrow n^{\text{top}} - n_l^{\text{top}}$ 
12:  $n_l^{\text{bot}} \leftarrow n_l - n_l^{\text{top}}$ ,  $n_r^{\text{bot}} \leftarrow n_r - n_r^{\text{top}}$ 
13:  $\mathcal{V}^{\text{top}} \leftarrow (a, 2s, n^{\text{top}}, n_l^{\text{top}}, n_r^{\text{top}})$ 
14:  $\mathcal{V}^{\text{bot}} \leftarrow (a+s, 2s, n^{\text{bot}}, n_l^{\text{bot}}, n_r^{\text{bot}})$ 
15:  $C_{\text{cur}} \leftarrow \text{OFRASSIGN}(T, \mathcal{V})$ 
16:  $C_{\text{top}} \leftarrow \text{OFRCONTROLBITS}(T, \mathcal{V}^{\text{top}})$ 
17:  $C_{\text{bot}} \leftarrow \text{OFRCONTROLBITS}(T, \mathcal{V}^{\text{bot}})$ 
18: return  $C_{\text{cur}} \parallel C_{\text{top}} \parallel C_{\text{bot}}$ 

```

Algorithm 10 OForkCtrl($x, y, t^{\text{top}}, t^{\text{bot}}$)

Input: Secret input tags x, y and target output tags $t^{\text{top}}, t^{\text{bot}}$.

Output: Secret two-bit OFORK control c .

```

1:  $z_{\text{straight}} \leftarrow \llbracket t^{\text{top}} = x \rrbracket \wedge \llbracket t^{\text{bot}} = y \rrbracket$ 
2:  $z_{\text{cross}} \leftarrow \llbracket t^{\text{top}} = y \rrbracket \wedge \llbracket t^{\text{bot}} = x \rrbracket$ 
3:  $z_x \leftarrow \llbracket x = 11 \rrbracket \wedge \llbracket y = 00 \rrbracket$ 
4:  $z_y \leftarrow \llbracket x = 00 \rrbracket \wedge \llbracket y = 11 \rrbracket$ 
5:  $z_{00,11} \leftarrow z_x \vee z_y$ 
6:  $z_{l,r} \leftarrow \llbracket t^{\text{top}} = 10 \rrbracket \wedge \llbracket t^{\text{bot}} = 01 \rrbracket$ 
7:  $z_{r,l} \leftarrow \llbracket t^{\text{top}} = 01 \rrbracket \wedge \llbracket t^{\text{bot}} = 10 \rrbracket$ 
8:  $z_{\text{dup}} \leftarrow z_{00,11} \wedge (z_{l,r} \vee z_{r,l})$ 
9:  $c_{\text{dup}} \leftarrow (z_y, z_x)$  ▷ Duplicate x for 00, y for 11
10:  $u_{\text{cross}} \leftarrow z_{\text{cross}} \wedge \neg z_{\text{straight}}$ 
11:  $u_{\text{dup}} \leftarrow z_{\text{dup}} \wedge \neg z_{\text{straight}} \wedge \neg z_{\text{cross}}$ 
12:  $c \leftarrow 01$ 
13:  $c \leftarrow \text{OSELECT}(c, 10, u_{\text{cross}})$ 
14:  $c \leftarrow \text{OSELECT}(c, c_{\text{dup}}, u_{\text{dup}})$ 
15: return  $c$ 

```

exact-count property is the invariant used by the recursive responsibility assignment described next.

4.3.2. Responsibility Assignment. Figure 4 gives an alternative representation of the same OFR network in Figure 2. While Figure 2 emphasizes the fixed butterfly-style wiring, Figure 4 logically groups the wires belonging to the same recursive subnetwork into shaded regions, making the recursive responsibility assignment explicit. This regrouping is only a logical view of the same network and introduces no additional routing operation.

For every OFORK in the current layer, we regard its left output as entering the *Top* subnetwork and its right output as entering the *Bottom* subnetwork. Top and Bottom

Algorithm 11 OFRAPPLY($D, C, \mathcal{V}, p$)

Input: Mutable data array D , secret control array C , public recursive view $\mathcal{V}$, and read position p .

Output: Next read position p .

```
1:  $(a, s, n, n_l, n_r) \leftarrow \mathcal{V}$ 
2: if  $n_l = 0$  or  $n_r = 0$  then
3:   return  $p$ 
4: end if
5: if  $n = 2$  then
6:    $(x, y) \leftarrow (D[a], D[a + s])$ 
7:    $(D[a], D[a + s]) \leftarrow \text{OFORK}(x, y, C[p])$ 
8:   return  $p + 1$ 
9: end if
10:  $\ell_g \leftarrow \lfloor n/2 \rfloor$ 
11:  $n^{\text{top}} \leftarrow \lceil n/2 \rceil, n^{\text{bot}} \leftarrow n - n^{\text{top}}$ 
12:  $n_l^{\text{top}} \leftarrow \lceil n_l/2 \rceil, n_r^{\text{top}} \leftarrow n^{\text{top}} - n_l^{\text{top}}$ 
13:  $n_l^{\text{bot}} \leftarrow n_l - n_l^{\text{top}}, n_r^{\text{bot}} \leftarrow n_r - n_r^{\text{top}}$ 
14:  $\mathcal{V}^{\text{top}} \leftarrow (a, 2s, n^{\text{top}}, n_l^{\text{top}}, n_r^{\text{top}})$ 
15:  $\mathcal{V}^{\text{bot}} \leftarrow (a + s, 2s, n^{\text{bot}}, n_l^{\text{bot}}, n_r^{\text{bot}})$ 
16: for  $i \leftarrow 0 \dots \ell_g - 1$  do
17:    $j_0 \leftarrow a + 2 * i * s, j_1 \leftarrow j_0 + s$ 
18:    $\text{OFORK}(D[j_0], D[j_1], C[p + i])$ 
19: end for
20:  $p \leftarrow p + \ell_g$ 
21:  $p \leftarrow \text{OFRAPPLY}(D, C, \mathcal{V}^{\text{top}}, p)$ 
22:  $p \leftarrow \text{OFRAPPLY}(D, C, \mathcal{V}^{\text{bot}}, p)$ 
23: return  $p$ 
```

describe where an output is processed in the next recursive step, whereas left and right responsibilities describe the final output region that a record must reach. Hence, a left or right responsibility may be assigned to either Top or Bottom at an intermediate layer and continue to be routed within that subnetwork.

The purpose of responsibility assignment is to determine what the two outputs of each OFORK should carry. For an input pair (x, y) , we denote the desired output tags by t^{top} and t^{bot} . Once these two tags are fixed, the corresponding two-bit OFORK control can be derived from the input pair and the desired outputs: the gate performs straight routing, cross routing, or duplicates one of its inputs. Thus, control generation is reduced to determining a valid responsibility assignment for the Top and Bottom outputs of every gate.

This assignment is performed recursively. For each current subnetwork, the public network structure determines in advance how many left and right responsibilities its Top and Bottom children must receive. The current layer assigns its input responsibilities to satisfy these targets, and the same process is then applied independently to the two children. Figure 4 illustrates this process for $n = 8$ and $n_l = n_r = 4$: after the first layer, the responsibilities are distributed to two smaller subnetworks, each of which is further divided in the next layer. Repeating this process progressively refines subnetwork-level responsibility counts into position-level routing decisions, until the output tags become a contiguous sequence of left-only tags followed by right-only tags.

To determine a valid assignment within one layer, we exploit the fact that each input tag belongs to $\{00, 10, 01, 11\}$. Therefore, a single OFORK has only ten possible unordered input-tag pairs. Table 1 enumerates these cases and classifies them according to the freedom they provide when assigning responsibilities to Top and Bottom. We represent the classification by (o, a_l, a_r, f_l, f_r) . Here, f_l and f_r denote left and right responsibilities whose contribution to Top is fixed, while a_l and a_r denote responsibilities that can be assigned to either subnetwork. The indicator o captures the special pair $\{10, 01\}$, for which assigning the left responsibility to Top necessarily assigns the right responsibility to Bottom, or vice versa.

This classification also captures duplication naturally. For example, the pair $\{11, 00\}$ contains one record carrying both responsibilities and one record carrying none. Its two responsibilities may remain together or be separated between Top and Bottom by duplicating the 11-tagged record. In contrast, the two responsibilities in $\{10, 01\}$ belong to different records and therefore form a coupled routing choice. By reducing all ten input cases to these assignment roles, the layer-wide problem becomes selecting how many available choices of each type should contribute to Top.

OFRASSIGN (Algorithm 8) performs this selection. It first determines the responsibility targets of the Top subnetwork, taking into account the unpaired bypass input when the current size is odd. It then scans all input pairs using Table 1 to summarize the fixed and adjustable assignment choices available in the layer. From these counts, OFRASSIGN determines a feasible number of choices of each type that must be assigned to Top. A second fixed-order scan realizes these choices on individual pairs, producing t^{top} and t^{bot} for every OFORK. OForkCtrl then translates the resulting target tags into the corresponding two-bit control word.

After the current layer is processed, the Top and Bottom subnetworks contain exactly their prescribed numbers of left and right responsibilities. OFRCONTROLBITS (Algorithm 9) therefore applies the same procedure recursively to both children. A subnetwork that requires only one output direction needs no further routing, while a mixed two-position subnetwork is resolved by a single OFORK. Repeating this process determines the desired outputs, and hence the control word, of every OFORK in the OFR network.

4.3.3. Online Apply. Once the controls have been generated, online execution no longer needs the routing tags or performs any responsibility assignment. OFRAPPLY (Algorithm 11) simply replays the precomputed controls on the payload array following the same public recursive order used during control generation: the current layer, followed by the Top and Bottom subnetworks. Accordingly, the control array is stored as $C_{\text{cur}} \parallel C_{\text{top}} \parallel C_{\text{bot}}$. Only the public subnetwork parameters are needed to determine which controls and payload positions are processed at each recursive call.

The responsibility-assignment discussion above logically groups the Top and Bottom subnetworks into separate regions for clarity. OFRAPPLY realizes the same structure

Algorithm 12 FFOS-FR(D, n, m, k)

Input: Mutable secret data array D of public length n ;
public $1 \leq m < n$ and $k > 1$.

Output: k independent uniform size- m samples.

Offline phase

```
1:  $M \leftarrow \text{MARKSAMPLES}(n, m, k)$ 
2: if  $mk > n$  then
3:    $F \leftarrow \text{BUILDFRONTIER}(n, m, k)$ 
4: else
5:    $F \leftarrow \perp$ 
6: end if
7:  $C \leftarrow \text{FFOSFRCONTROLBITS}(M, F, n, m, k)$ 
```

Online phase

```
8:  $(\mathcal{S}, p) \leftarrow \text{FFOSFRAPPLY}(D, C, F, n, m, k, (0, 0))$ 
9: assert  $p = (|C_{\text{cmp}}|, |C_{\text{ofr}}|)$ 
10: return  $\mathcal{S}$ 
```

without physically separating or merging the payload array. A logical subnetwork is represented by the public view $\mathcal{V} = (a, s, n, n_l, n_r)$, where its j -th logical position is stored at $a + js$. After the current layer is applied, the left output of each OFORK belongs to the Top subnetwork and the right output belongs to the Bottom subnetwork. These two subnetworks are therefore represented by strided views with address parameters $(a, 2s)$ and $(a + s, 2s)$, respectively.

Because both views refer to the same backing array, every OFORK can update its publicly determined pair in place, and the recursive execution requires neither auxiliary payload arrays nor an explicit merge step. Thus, the offline phase determines all secret routing decisions, while the online phase consists only of a fixed sequence of OFORK operations over public addresses. Section 4.4 next integrates OFR into the FFOS sampling tree.

4.4. FFOS-FR Implementation

FFOS-FR integrates OFR into the hierarchical sampling framework of FFOS-C. As shown in Algorithm 12, it retains the same membership generation and sampling-tree structure, but changes how candidate arrays are split once their capacities become exact-fit.

The key observation is that the frontier introduced in Section 3.4 forms a natural transition point between compaction and OFR. Each frontier group containing k_v samples is materialized with public length mk_v and is therefore an exact-fit node. As established in Section 4.1, the exact-fit property is preserved by every descendant. Consequently, after the frontier candidate arrays have been formed, all subsequent internal nodes can be processed by OFR. Compaction is therefore needed only to materialize the initial exact-fit candidate groups, rather than at every level of the sampling tree.

This gives FFOS-FR a simple two-stage structure. Before entering an exact-fit subtree, the required candidate group is materialized using the same oblivious-compaction mechanism as FFOS-C. If $mk > n$, BUILDFRONTIER partitions the samples into several such groups; if $mk < n$,

a single candidate array of length mk is materialized. When $mk = n$, the root is already exact-fit and this materialization step is unnecessary. From that point onward, every internal node uses a single OFR instance to jointly produce its two child regions, replacing the two independent compactations required by FFOS-C.

Offline, FFOS-FR derives the routing tags of each exact-fit node from the membership words, generates the corresponding OFR controls, and replays them on the membership array so that the offline membership state remains positionally aligned with the future online payload state. Online execution follows the same public structure: any initial candidate-group materialization is performed using the precomputed compaction controls, after which the remaining recursive subtree is processed entirely by OFRAPPLY.

Thus, the benefit of FFOS-FR is not limited to isolated exact-fit nodes. Once the execution reaches the frontier, OFR replaces the two-compaction split throughout the entire remaining sampling tree. Detailed control generation and online execution are provided in Appendix A.

4.5. Analysis

5. Experiments

6. Conclusion

Acknowledgments

The authors would like to thank...

References

- [1] Y. Xu, W. Cui, and M. Peinado, “Controlled-channel attacks: Deterministic side channels for untrusted operating systems,” in *2015 IEEE Symposium on Security and Privacy (SP)*, 2015.
- [2] J. Van Bulck, N. Weichbrodt, R. Kapitza, F. Piessens, and R. Strackx, “Telling your secrets without page faults: Stealthy page table-based attacks on enclaved execution,” in *26th USENIX Security Symposium (USENIX Security 17)*, 2017.
- [3] S. Sasy, A. Johnson, and I. Goldberg, “Fast Fully Oblivious Compaction and Shuffling,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.
- [4] T. Gu, Y. Wang, A. Tinoco, B. Chen, K. Yi, and E. Shi, “Flexway O-Sort: Enclave-Friendly and Optimal Oblivious Sorting,” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025.
- [5] P. Mishra, R. Poddar, J. Chen, A. Chiesa, and R. A. Popa, “Obliv: An efficient oblivious search index,” in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018.
- [6] O. Ohrimenko, F. Schuster, C. Fournet, A. Mehta, S. Nowozin, K. Vaswani, and M. Costa, “Oblivious multi-party machine learning on trusted processors,” in *25th USENIX Security Symposium (USENIX Security 16)*, 2016.
- [7] J. Bater, Y. Park, X. He, X. Wang, and J. Rogers, “SAQE: Practical Privacy-Preserving Approximate Query Processing for Data Federations,” *Proceedings of the VLDB Endowment*, 2020.
- [8] Q. Luo, Y. Wang, K. Yi, S. Wang, and F. Li, “Secure Sampling for Approximate Multi-party Query Processing,” *Proceedings of the ACM on Management of Data*, 2023.

- [9] S. Sasy and O. Ohrimenko, "Oblivious Sampling Algorithms for Private Data Analysis," in *Advances in Neural Information Processing Systems*, 2019.
- [10] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep Learning with Differential Privacy," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.
- [11] M. T. Goodrich, "Data-Oblivious External-Memory Algorithms for the Compaction, Selection, and Sorting of Outsourced Data," in *Proceedings of the Twenty-Third Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 2011.
- [12] S. Sasy, A. Johnson, and I. Goldberg, "Waks-On/Waks-Off: Fast Oblivious Offline/Online Shuffling and Sorting with Waksman Networks," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.
- [13] A. Mavrogianakis, X. Wang, I. Demertzis, D. Papadopoulos, and M. Garofalakis, "OBLIVIATOR: OBLIVIOUS Parallel Joins and other OPERATORS in Shared Memory Environments," in *34th USENIX Security Symposium (USENIX Security)*, 2025.
- [14] G. Asharov, T.-H. H. Chan, K. Nayak, R. Pass, L. Ren, and E. Shi, "Bucket Oblivious Sort: An Extremely Simple Oblivious Sort," in *Symposium on Simplicity in Algorithms (SOSA 2020)*, 2020.
- [15] O. Goldreich and R. Ostrovsky, "Software protection and simulation on oblivious RAMs," *Journal of the ACM*, 1996.
- [16] E. Stefanov, M. van Dijk, E. Shi, C. Fletcher, L. Ren, X. Yu, and S. Devadas, "Path ORAM: An extremely simple oblivious RAM protocol," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2013.
- [17] X. S. Wang, K. Nayak, C. Liu, T.-H. H. Chan, E. Shi, E. Stefanov, and Y. Huang, "Oblivious data structures," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2014.
- [18] X. Wang, H. Chan, and E. Shi, "Circuit ORAM: On tightness of the Goldreich-Ostrovsky lower bound," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015.
- [19] G. Asharov, I. Komargodski, W.-K. Lin, K. Nayak, E. Peserico, and E. Shi, "OptORAMa: Optimal oblivious RAM," in *Advances in Cryptology—EUROCRYPT 2020, Part II*, 2020.
- [20] E. Shi, "Path oblivious heap: Optimal and practical oblivious priority queue," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020.
- [21] S. Lee, M.-W. Shih, P. Gera, T. Kim, H. Kim, and M. Peinado, "Inferring fine-grained control flow inside SGX enclaves with branch shadowing," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017.
- [22] W. Wang, G. Chen, X. Pan, Y. Zhang, X. Wang, V. Bindschaedler, H. Tang, and C. A. Gunter, "Leaky cauldron on the dark land: Understanding memory side-channel hazards in SGX," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017.
- [23] S. Sasy, S. Gorbunov, and C. W. Fletcher, "ZeroTrace: Oblivious memory primitives from intel SGX," in *Proceedings of the 2018 Network and Distributed System Security Symposium (NDSS)*, 2018.
- [24] A. Ahmad, K. Kim, M. I. Sarfaraz, and B. Lee, "OBLIViate: A data oblivious filesystem for intel SGX," in *Proceedings of the 2018 Network and Distributed System Security Symposium (NDSS)*, 2018.
- [25] A. Tinoco, S. Gao, and E. Shi, "EnigMap: External-memory oblivious map for secure enclaves," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023.
- [26] C. Liu, X. S. Wang, K. Nayak, Y. Huang, and E. Shi, "ObliVM: A programming framework for secure computation," in *2015 IEEE Symposium on Security and Privacy (SP)*, 2015.
- [27] O. Ohrimenko, M. Costa, C. Fournet, C. Gkantsidis, M. Kohlweiss, and D. Sharma, "Observing and preventing leakage in MapReduce," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015.
- [28] K. E. Batchier, "Sorting Networks and Their Applications," in *Proceedings of the AFIPS Spring Joint Computer Conference*, 1968.
- [29] A. Waksman, "A Permutation Network," *Journal of the ACM*, 1968.
- [30] M. Ajtai, J. Komlós, and E. Szemerédi, "An $O(n \log n)$ Sorting Network," in *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing (STOC)*, 1983.
- [31] M. T. Goodrich, "Zig-zag Sort: A Simple Deterministic Data-Oblivious Sorting Algorithm Running in $O(n \log n)$ Time," in *Proceedings of the Forty-Sixth Annual ACM Symposium on Theory of Computing (STOC)*, 2014.
- [32] O. Ohrimenko, M. T. Goodrich, R. Tamassia, and E. Upfal, "The Melbourne Shuffle: Improving Oblivious Storage in the Cloud," in *Automata, Languages, and Programming: 41st International Colloquium, ICALP 2014, Proceedings, Part II*, 2014.
- [33] S. Patel, G. Persiano, and K. Yeo, "CacheShuffle: A Family of Oblivious Shuffles," in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, 2018.
- [34] W. Holland, O. Ohrimenko, and A. Wirth, "Efficient Oblivious Permutation via the Waksman Network," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security (AsiaCCS)*, 2022.
- [35] N. Ngai, I. Demertzis, J. G. Chamani, and D. Papadopoulos, "Distributed & Scalable Oblivious Sorting and Shuffling," in *2024 IEEE Symposium on Security and Privacy (S&P)*, 2024.
- [36] J. S. Vitter, "Random Sampling with a Reservoir," *ACM Transactions on Mathematical Software*, 1985.
- [37] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, 1979.
- [38] Q. Luo, Q. Xu, and C. Yang, "Secure Multi-Party Sampling over Joins," *Proceedings of the VLDB Endowment*, 2026.
- [39] A. Bittau, Ú. Erlingsson, P. Maniatis, I. Mironov, A. Raghunathan, D. Lie, M. Rudominer, U. Kode, J. Tinnes, and B. Seefeld, "Prochlo: Strong Privacy for Analytics in the Crowd," in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017.
- [40] B. Balle, G. Barthe, and M. Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences," in *Advances in Neural Information Processing Systems*, 2018.
- [41] Y.-X. Wang, B. Balle, and S. P. Kasiviswanathan, "Subsampled Rényi Differential Privacy and Analytical Moments Accountant," in *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2019.
- [42] T. Murakami, Y. Sei, and R. Eriguchi, "Fully Oblivious Differential Privacy for Frequency Estimation in the Augmented Shuffle Model with Trusted Processors," in *35th USENIX Security Symposium (USENIX Security)*, 2026.
- [43] W. Zheng, A. Dave, J. G. Beekman, R. A. Popa, J. E. Gonzalez, and I. Stoica, "Opaque: An Oblivious and Encrypted Distributed Analytics Platform," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017.
- [44] S. Eskandarian and M. Zaharia, "ObliDB: Oblivious Query Processing for Secure Databases," *Proceedings of the VLDB Endowment*, 2019.
- [45] S. Krastnikov, F. Kerschbaum, and D. Stebila, "Efficient Oblivious Database Joins," *Proceedings of the VLDB Endowment*, 2020.
- [46] S. Chu, D. Zhuo, E. Shi, and T.-H. H. Chan, "Differentially Oblivious Database Joins: Overcoming the Worst-Case Curse of Fully Oblivious Algorithms," in *2nd Conference on Information-Theoretic Cryptography (ITC)*, 2021.

- [47] E. Baum, S. Buxbaum, N. Mathai, M. Faisal, V. Kalavri, M. Varia, and J. Liagouris, “ORQ: Complex Analytics on Private Data with Strong Security Guarantees,” in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*, 2025.
- [48] T.-H. H. Chan, K.-M. Chung, B. M. Maggs, and E. Shi, “Foundations of Differentially Oblivious Algorithms,” in *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2019.
- [49] S. Agarwal, B. Mozafari, A. Panda, H. Milner, S. Madden, and I. Stoica, “BlinkDB: Queries with Bounded Errors and Bounded Response Times on Very Large Data,” in *Proceedings of the 8th ACM European Conference on Computer Systems (EuroSys)*, 2013.
- [50] J. Allen, B. Ding, J. Kulkarni, H. Nori, O. Ohrimenko, and S. Yekhanin, “An Algorithmic Framework For Differentially Private Data Analysis on Trusted Processors,” in *Advances in Neural Information Processing Systems*, 2019.

Appendix

Algorithm 13 FFOSFRCONTROLBITS(M, F, n, m, k)

Input: Secret membership array M ; public F, n, m, k .

Output: Control streams $C = (C_{\text{cmp}}, C_{\text{ofr}})$.

- 1: Create empty control arrays C_{cmp} and C_{ofr}
 - 2: FFOSFRWORK($M, C_{\text{cmp}}, C_{\text{ofr}}, F, n, m, k$)
 - 3: **return** $C = (C_{\text{cmp}}, C_{\text{ofr}})$
-

Algorithm 14 FFOSFRWORK($M, C_{\text{cmp}}, C_{\text{ofr}}, F, n, m, k$)

Input: Mutable membership array M , control streams $C_{\text{cmp}}, C_{\text{ofr}}$; public F, n, m, k .

Output: Append the controls of the current subtree.

- 1: **if** $k = 1$ **then**
- 2: **assert** $n = m$
- 3: **return**
- 4: **end if**
- 5: $k_l \leftarrow \lfloor k/2 \rfloor, k_r \leftarrow k - k_l$
- 6: $\triangleright$ Process an exact-fit node using OFR
- 7: **if** $n = mk$ **then**
- 8: $n_l \leftarrow mk_l, n_r \leftarrow mk_r$
- 9: $\mathcal{V}_0 \leftarrow (0, 1, n, n_l, n_r)$
- 10: $\mu_l \leftarrow (1 \ll k_l) - 1, \mu_r \leftarrow (1 \ll k_r) - 1$
- 11: Create a length- n two-bit routing-tag array T
- 12: **for** $i \leftarrow 0 \dots n - 1$ **do**
- 13: $T[i][0] \leftarrow \llbracket (M[i] \& \mu_l) \neq 0 \rrbracket$
- 14: $T[i][1] \leftarrow \llbracket ((M[i] \gg k_l) \& \mu_r) \neq 0 \rrbracket$
- 15: **end for**
- 16: OFRNORMALIZE(T, n, n_l, n_r)
- 17: $C_{\text{node}} \leftarrow \text{OFRCONTROLBITS}(T, \mathcal{V}_0)$
- 18: Append C_{node} to C_{ofr}
- 19: $p \leftarrow \text{OFRAPPLY}(M, C_{\text{node}}, \mathcal{V}_0, 0)$
- 20: **assert** $p = |C_{\text{node}}|$
- 21: $M_l \leftarrow M[0 : n_l], M_r \leftarrow M[n_l : n]$
- 22: $\triangleright$ Project memberships to the two child groups
- 23: **for** $i \leftarrow 0 \dots n_l - 1$ **do**
- 24: $M_l[i] \leftarrow M_l[i] \& \mu_l$
- 25: **end for**
- 26: **for** $i \leftarrow 0 \dots n_r - 1$ **do**
- 27: $M_r[i] \leftarrow (M_r[i] \gg k_l) \& \mu_r$

- 28: **end for**
 - 29: FFOSFRWORK($M_l, C_{\text{cmp}}, C_{\text{ofr}}, \perp, n_l, m, k_l$)
 - 30: FFOSFRWORK($M_r, C_{\text{cmp}}, C_{\text{ofr}}, \perp, n_r, m, k_r$)
 - 31: **return**
 - 32: **end if**
 - 33: $\triangleright$ Materialize exact-fit frontier groups
 - 34: **if** $F \neq \perp$ **then**
 - 35: $F' \leftarrow F$
 - 36: **else**
 - 37: $F' \leftarrow [(0, k)]$
 - 38: **end if**
 - 39: **for** $v \leftarrow 0 \dots |F'| - 1$ **do**
 - 40: $(s_v, k_v) \leftarrow F'[v]$
 - 41: $n_v \leftarrow \min(n, mk_v)$
 - 42: $\mu_v \leftarrow (1 \ll k_v) - 1$
 - 43: Copy M into M_{tmp}
 - 44: **if** $n_v < n$ **then**
 - 45: Create a length- n bit array C_v
 - 46: **for** $i \leftarrow 0 \dots n - 1$ **do**
 - 47: $C_v[i] \leftarrow \llbracket ((M[i] \gg s_v) \& \mu_v) \neq 0 \rrbracket$
 - 48: **end for**
 - 49: Append C_v to C_{cmp}
 - 50: $\text{OCompact}(M_{\text{tmp}}, C_v)$
 - 51: **end if**
 - 52: Create a length- n_v membership array M_v
 - 53: **for** $i \leftarrow 0 \dots n_v - 1$ **do**
 - 54: $M_v[i] \leftarrow (M_{\text{tmp}}[i] \gg s_v) \& \mu_v$
 - 55: **end for**
 - 56: FFOSFRWORK($M_v, C_{\text{cmp}}, C_{\text{ofr}}, \perp, n_v, m, k_v$)
 - 57: **end for**
 - 58: **return**
-

Algorithm 15 FFOSFRAPPLY(D, C, F, n, m, k, p)

Input: Mutable data array D , control streams C ; public F, n, m, k and read position p

Output: Sample list $\mathcal{S}$ and next read position p

- 1: $(C_{\text{cmp}}, C_{\text{ofr}}) \leftarrow C$
- 2: $(p_{\text{cmp}}, p_{\text{ofr}}) \leftarrow p$
- 3: **if** $k = 1$ **then**
- 4: **assert** $n = m$
- 5: Copy $D[0 : m]$ into $\mathcal{S}$
- 6: **return** $(\mathcal{S}, p)$
- 7: **end if**
- 8: $k_l \leftarrow \lfloor k/2 \rfloor, k_r \leftarrow k - k_l$
- 9: $\triangleright$ Process an exact-fit node using OFR
- 10: **if** $n = mk$ **then**
- 11: $n_l \leftarrow mk_l, n_r \leftarrow mk_r$
- 12: $\mathcal{V}_0 \leftarrow (0, 1, n, n_l, n_r)$
- 13: $p_{\text{ofr}} \leftarrow \text{OFRAPPLY}(D, C_{\text{ofr}}, \mathcal{V}_0, p_{\text{ofr}})$
- 14: $p \leftarrow (p_{\text{cmp}}, p_{\text{ofr}})$
- 15: $D_l \leftarrow D[0 : n_l], D_r \leftarrow D[n_l : n]$
- 16: $(\mathcal{S}_l, p) \leftarrow \text{FFOSFRAPPLY}(D_l, C, \perp, n_l, m, k_l, p)$
- 17: $(\mathcal{S}_r, p) \leftarrow \text{FFOSFRAPPLY}(D_r, C, \perp, n_r, m, k_r, p)$
- 18: **return** $(\mathcal{S}_l \parallel \mathcal{S}_r, p)$
- 19: **end if**
- 20: $\triangleright$ Materialize and process frontier groups
- 21: **if** $F \neq \perp$ **then**

```

20:    $F' \leftarrow F$ 
21: else
22:    $F' \leftarrow [(0, k)]$ 
23: end if
24:  $\mathcal{S} \leftarrow []$ 
25: for  $v \leftarrow 0 \dots |F'| - 1$  do
26:    $(s_v, k_v) \leftarrow F'[v]$ 
27:    $n_v \leftarrow \min(n, mk_v)$ 
28:   Copy  $D$  into  $D_{\text{tmp}}$ 
29:    $(p_{\text{cmp}}, p_{\text{ofr}}) \leftarrow p$ 
30:   if  $n_v < n$  then
31:      $C_v \leftarrow C_{\text{cmp}}[p_{\text{cmp}} : p_{\text{cmp}} + n)$ 
32:      $p_{\text{cmp}} \leftarrow p_{\text{cmp}} + n$ 
33:      $p \leftarrow (p_{\text{cmp}}, p_{\text{ofr}})$ 
34:      $\text{OCompact}(D_{\text{tmp}}, C_v)$ 
35:   end if
36:    $D_v \leftarrow D_{\text{tmp}}[0 : n_v)$ 
37:    $(\mathcal{S}_v, p) \leftarrow \text{FFOSFRAPPLY}(D_v, C, \perp, n_v, m, k_v, p)$ 
38:    $\mathcal{S} \leftarrow \mathcal{S} \parallel \mathcal{S}_v$ 
39: end for
40: return  $(\mathcal{S}, p)$ 

```
